

Python Lab programs

1. Solve the Tic-Tac-Toe problem using the Depth First Search technique.

TIC-TAC-TOE USING DFS

```
def print_board(b):
    for i in range(0, 9, 3):
        print(b[i], "|", b[i+1], "|", b[i+2])
    print()

def winner(b, p):
    wins = [(0,1,2),(3,4,5),(6,7,8),
            (0,3,6),(1,4,7),(2,5,8),
            (0,4,8),(2,4,6)]
    return any(b[x]==b[y]==b[z]==p for x,y,z in wins)

def dfs(b, maxi):
    if winner(b, "X"): return 1
    if winner(b, "O"): return -1
    if " " not in b: return 0

    best = -100 if maxi else 100

    for i in range(9):
        if b[i] == " ":
            b[i] = "X" if maxi else "O"
            score = dfs(b, not maxi)
            b[i] = " "

            best = max(best, score) if maxi else min(best, score)
```

```
return best
```

```
def best_move(b):
```

```
    best, move = -100, -1
```

```
    for i in range(9):
```

```
        if b[i] == " ":
```

```
            b[i] = "X"
```

```
            score = dfs(b, False)
```

```
            b[i] = " "
```

```
        if score > best:
```

```
            best, move = score, i
```

```
    return move
```

```
# MAIN PROGRAM
```

```
board = [" "] * 9
```

```
while True:
```

```
    print_board(board)
```

```
    p = int(input("Enter position (0-8): "))
```

```
    if board[p] != " ":
```

```
        print("Already filled!")
```

```
        continue
```

```
    board[p] = "O"
```

```
if winner(board, "O"):
    print_board(board)
    print("Player Wins!")
    break
```

```
if " " not in board:
    print("Draw!")
    break
```

```
c = best_move(board)
board[c] = "X"
```

```
print("Computer chose:", c)
```

```
if winner(board, "X"):
    print_board(board)
    print("Computer Wins!")
    break
```

```
if " " not in board:
    print_board(board)
    print("Draw!")
    break
```

- 2. Show that the 8-puzzle states are divided into two disjoint sets, such that any state is reachable from any other state in the same set, while no state is reachable from any state in the other set.**

```
# =====
```

```
# 8-PUZZLE DISJOINT SET CHECK
```

```
# USER INPUT VERSION
```

```
# =====
```

```
# Example Input:
```

```
# 1 2 3 4 5 6 7 8 0
```

```
# =====
```

```
# FUNCTION TO COUNT INVERSIONS
```

```
# =====
```

```
def count_inversions(state):
```

```
    inversions = 0
```

```
    # Remove blank tile (0)
```

```
    puzzle = []
```

```
    for value in state:
```

```
        if value != 0:
```

```
            puzzle.append(value)
```

```
    # Count inversions
```

```
    for i in range(len(puzzle)):
```

```
        for j in range(i + 1, len(puzzle)):
```

```
    if puzzle[i] > puzzle[j]:
```

```
        inversions += 1
```

```
    return inversions
```

```
# =====
```

```
# FUNCTION TO GET PARITY
```

```
# =====
```

```
def get_parity(state):
```

```
    return count_inversions(state) % 2
```

```
# =====
```

```
# CHECK SOLVABILITY
```

```
# =====
```

```
def is_solvable(state):
```

```
    return get_parity(state) == 0
```

```
# =====
```

```
# DISPLAY PUZZLE STATE
```

```
# =====
```

```
def display_state(state):
```

```
    print()
```

```
    for i in range(3):
```

```
        row = []
```

```
        for j in range(3):
```

```
            value = state[i * 3 + j]
```

```
            if value == 0:
```

```
                row.append(" ")
```

```
            else:
```

```
                row.append(str(value))
```

```
        print(" ".join(row))
```

```
    print()
```

```
# =====
```

```
# USER INPUT
```

```
# =====
```

```
print("===== 8-PUZZLE CHECK =====\n")
```

```
print("Enter State 1")
```

```
print("Example: 1 2 3 4 5 6 7 8 0\n")
```

```
state1 = list(map(int, input("State 1: ").split()))
```

```
print("\nEnter State 2")
```

```
print("Example: 1 2 3 4 5 6 0 7 8\n")
```

```
state2 = list(map(int, input("State 2: ").split()))
```

```
# =====
```

```
# DISPLAY STATES
```

```
# =====
```

```
print("\n===== STATE 1 =====")
```

```
display_state(state1)
```

```
print("===== STATE 2 =====")
```

```
display_state(state2)
```

```

# =====

# CHECK SOLVABILITY

# =====

solvable1 = is_solvable(state1)
solvable2 = is_solvable(state2)


# =====

# DISPLAY SOLVABILITY

# =====

print("State 1 Inversions :", count_inversions(state1))
print("State 2 Inversions :", count_inversions(state2))

print()

if solvable1:

    print("State 1 is Solvable")

else:

    print("State 1 is Unsolvable")

if solvable2:

```



```
print("State 2 is Solvable")
```

```
else:
```

```
print("State 2 is Unsolvable")
```

```
# =====
```

```
# CHECK DISJOINT SETS
```

```
# =====
```

```
print("\n===== RESULT =====\n")
```

```
if solvable1 == solvable2:
```

```
    print("Both states are in the SAME set")
```

```
    print("and are reachable from each other.")
```

```
else:
```

```
    print("The states belong to DIFFERENT sets")
```

```
    print("and are NOT reachable from each other.")
```

```
# =====
```

```
# THEORY
```

```
# =====
```

```
print("\n===== THEORY =====\n")
```

```
print("8-puzzle states are divided into")
```

```
print("two disjoint sets based on inversion parity.\n")
```

```
print("Even inversions -> Solvable")
```

```
print("Odd inversions -> Unsolvable")
```

```
# =====
```

```
# END OF PROGRAM
```

```
# =====
```

3. To represent and evaluate different scenarios using predicate logic and knowledge rules.

```
# =====
```

```
# PREDICATE LOGIC AND KNOWLEDGE RULES
```

```
# =====
```

```
# Scenario:
```

```
# Determine whether a person can vote
```

```
# Rule:
```

```
# IF person age >= 18
```

```
# THEN person is eligible to vote
```

```
# =====  
# KNOWLEDGE BASE  
# =====
```

```
people = {
```

```
    "Ravi": 22,
```

```
    "Anu": 17,
```

```
    "Kiran": 19,
```

```
    "Meena": 15
```

```
}
```

```
# =====  
# PREDICATE FUNCTION  
# =====
```

```
def can_vote(age):
```

```
    return age >= 18
```

```
# =====  
# DISPLAY KNOWLEDGE BASE  
# =====
```

```
print("===== KNOWLEDGE BASE =====\n")
```

```
for person in people:
```

```
    print(person, "Age =", people[person])
```

```

# =====
# APPLY PREDICATE LOGIC RULE
# =====

print("\n===== PREDICATE LOGIC RESULT =====\n")

for person in people:

    age = people[person]

    # Predicate Rule Evaluation
    if can_vote(age):

        print(person, "CAN vote")

    else:

        print(person, "CANNOT vote")

# =====
# LOGIC REPRESENTATION
# =====

print("\n===== KNOWLEDGE RULE =====\n")

print("Rule:")
print("IF Age >= 18 THEN Eligible_to_Vote(Person)")

```

4. To apply the Find-S and Candidate Elimination algorithms to a concept learning task and compare their inductive biases and outputs.
data = [

```

['Youth', 'High', 'No', 'Fair', 'No'],
['Youth', 'High', 'No', 'Excellent', 'No'],
['Middle', 'High', 'No', 'Fair', 'Yes'],
['Senior', 'Medium', 'No', 'Fair', 'Yes'],
['Senior', 'Low', 'Yes', 'Fair', 'Yes'],
['Senior', 'Low', 'Yes', 'Excellent', 'No'],
['Middle', 'Low', 'Yes', 'Excellent', 'Yes'],
['Youth', 'Medium', 'No', 'Fair', 'No'],
['Youth', 'Low', 'Yes', 'Fair', 'Yes'],
['Senior', 'Medium', 'Yes', 'Fair', 'Yes']
]

# =====
# FIND-S ALGORITHM
# =====

def find_s(training_data):

    num_attributes = len(training_data[0]) - 1

    # Initialize most specific hypothesis
    hypothesis = ['0'] * num_attributes

    print("Initial Hypothesis:")
    print(hypothesis)

    # Consider only positive examples
    for example in training_data:

        if example[-1] == 'Yes':

            for i in range(num_attributes):

                if hypothesis[i] == '0':
                    hypothesis[i] = example[i]

                elif hypothesis[i] != example[i]:
                    hypothesis[i] = '?'

    return hypothesis

# =====
# CANDIDATE ELIMINATION ALGORITHM
# =====

```

```

def candidate_elimination(training_data):

    num_attributes = len(training_data[0]) - 1

    # Initialize Specific Hypothesis
    S = ['0'] * num_attributes

    # Initialize General Hypothesis
    G = [['?'] * num_attributes]

    print("\nInitial Specific Hypothesis:")
    print(S)

    print("\nInitial General Hypothesis:")
    print(G)

    # Process each training example
    for example in training_data:

        attributes = example[:-1]
        target = example[-1]

        # -----
        # POSITIVE EXAMPLE
        # -----
        if target == 'Yes':

            # Update Specific Hypothesis
            for i in range(num_attributes):

                if S[i] == '0':
                    S[i] = attributes[i]

                elif S[i] != attributes[i]:
                    S[i] = '?'

            # Remove inconsistent hypotheses from G
            G = [g for g in G if all(
                g[i] == '?' or g[i] == attributes[i]
                for i in range(num_attributes)
            )]

        # -----
        # NEGATIVE EXAMPLE
        # -----
        else:

```

```

        new_G = []

        for g in G:

            for i in range(num_attributes):

                if g[i] == '?':

                    if S[i] != attributes[i]:

                        new_hypothesis = g.copy()
                        new_hypothesis[i] = S[i]

                        new_G.append(new_hypothesis)

        G = new_G

    return S, G

# =====
# MAIN PROGRAM
# =====

print("=====")
print("FIND-S ALGORITHM")
print("=====")

final_hypothesis = find_s(data)

print("\nFinal Hypothesis from Find-S:")
print(final_hypothesis)

print("\n\n=====")
print("CANDIDATE ELIMINATION ALGORITHM")
print("=====")

S_final, G_final = candidate_elimination(data)

print("\nFinal Specific Hypothesis:")
print(S_final)

print("\nFinal General Hypothesis:")
for g in G_final:
    print(g)

```

5. To construct a decision tree using the ID3 algorithm on a simple classification dataset.

```
# =====
# ID3 DECISION TREE ALGORITHM
# DIFFERENT DATASET - STUDENT RESULT PREDICTION
# =====

# DATASET:
# Predict whether a student will PASS or FAIL
#
# Attributes:
# StudyHours, Attendance, Assignment
#
# Target:
# Result = Pass / Fail
# =====

import pandas as pd
import math

# =====
# TRAINING DATASET
# =====

data = {
    'StudyHours': ['High', 'High', 'Medium', 'Low', 'Low',
                  'Medium', 'High', 'Low', 'Medium', 'High'],
    'Attendance': ['Good', 'Poor', 'Good', 'Poor', 'Good',
                  'Good', 'Poor', 'Poor', 'Good', 'Good'],
    'Assignment': ['Yes', 'Yes', 'Yes', 'No', 'No',
                  'Yes', 'No', 'No', 'Yes', 'Yes'],
    'Result': ['Pass', 'Pass', 'Pass', 'Fail', 'Fail',
              'Pass', 'Fail', 'Fail', 'Pass', 'Pass']
}

df = pd.DataFrame(data)

print("=====")
print("TRAINING DATASET")
print("=====")

print(df)
```



```

# =====
# FUNCTION TO CALCULATE ENTROPY
# =====

def entropy(target_column):

    values = target_column.unique()

    entropy_value = 0

    for value in values:

        probability = len(target_column[target_column == value]) / len(target_column)

        entropy_value -= probability * math.log2(probability)

    return entropy_value

# =====
# FUNCTION TO CALCULATE INFORMATION GAIN
# =====

def information_gain(data, attribute, target_name):

    # Total entropy
    total_entropy = entropy(data[target_name])

    values = data[attribute].unique()

    weighted_entropy = 0

    # Calculate weighted entropy
    for value in values:

        subset = data[data[attribute] == value]

        probability = len(subset) / len(data)

        weighted_entropy += probability * entropy(subset[target_name])

    gain = total_entropy - weighted_entropy

    return gain

# =====

```

```
# ID3 ALGORITHM
```

```
# =====
```

```
def id3(data, original_data, features, target_name):
```

```
    # If all target values are same
```

```
    if len(data[target_name].unique()) == 1:
```

```
        return data[target_name].unique()[0]
```

```
    # If dataset is empty
```

```
    elif len(data) == 0:
```

```
        return original_data[target_name].mode()[0]
```

```
    # If no features left
```

```
    elif len(features) == 0:
```

```
        return data[target_name].mode()[0]
```

```
    else:
```

```
        gains = []
```

```
        # Calculate gain for each feature
```

```
        for feature in features:
```

```
            gains.append(information_gain(data, feature, target_name))
```

```
        # Best feature selection
```

```
        best_feature = features[gains.index(max(gains))]
```

```
        # Create decision tree
```

```
        tree = {best_feature: {}}
```

```
        # Remaining features
```

```
        remaining_features = [f for f in features if f != best_feature]
```

```
        # Build subtree
```

```
        for value in data[best_feature].unique():
```

```
            subset = data[data[best_feature] == value]
```

```
            subtree = id3(subset,  
                          original_data,  
                          remaining_features,  
                          target_name)
```

```
            tree[best_feature][value] = subtree
```

```

        return tree

# =====
# MAIN PROGRAM
# =====

features = list(df.columns[:-1])

decision_tree = id3(df, df, features, 'Result')

print("\n=====")
print("GENERATED DECISION TREE")
print("=====")

print(decision_tree)

```

6. To assess how the ID3 algorithm performs on datasets with varying characteristics and complexity, examining overfitting, underfitting, and decision tree depth.

```

# -----
# IMPORT LIBRARIES
# -----

import math
import random
from collections import Counter
from copy import deepcopy

# =====
# ENTROPY FUNCTION
# =====

def entropy(data):

    labels = [row[-1] for row in data]

    total = len(labels)

    counts = Counter(labels)

    ent = 0

    for count in counts.values():

        probability = count / total

```

```

        ent -= probability * math.log2(probability)

    return ent

# =====
# INFORMATION GAIN
# =====

def information_gain(data, feature_index):

    total_entropy = entropy(data)

    feature_values = {}

    for row in data:

        value = row[feature_index]

        if value not in feature_values:
            feature_values[value] = []

        feature_values[value].append(row)

    weighted_entropy = 0

    total = len(data)

    for subset in feature_values.values():

        weighted_entropy += (len(subset) / total) * entropy(subset)

    gain = total_entropy - weighted_entropy

    return gain

# =====
# MAJORITY CLASS
# =====

def majority_class(data):

    labels = [row[-1] for row in data]

    return Counter(labels).most_common(1)[0][0]

```

```

# =====
# ID3 ALGORITHM
# =====

def id3(data, features, depth=0, max_depth=None):

    labels = [row[-1] for row in data]

    # If all labels are same
    if labels.count(labels[0]) == len(labels):
        return labels[0]

    # If no features left
    if len(features) == 0:
        return majority_class(data)

    # Depth control
    if max_depth is not None and depth >= max_depth:
        return majority_class(data)

    # Find best feature
    gains = []

    for feature in features:
        gains.append(information_gain(data, feature))

    best_feature = features[gains.index(max(gains))]

    tree = {best_feature: {}}

    # Get unique values
    values = set([row[best_feature] for row in data])

    for value in values:

        subset = []

        for row in data:
            if row[best_feature] == value:
                subset.append(row)

        if len(subset) == 0:
            tree[best_feature][value] = majority_class(data)

```

```

    else:
        remaining_features = deepcopy(features)
        remaining_features.remove(best_feature)

        subtree = id3(
            subset,
            remaining_features,
            depth + 1,
            max_depth
        )

        tree[best_feature][value] = subtree

    return tree

# =====
# PREDICTION FUNCTION
# =====

def predict(tree, sample):

    if not isinstance(tree, dict):
        return tree

    feature = list(tree.keys())[0]

    value = sample[feature]

    if value in tree[feature]:
        return predict(tree[feature][value], sample)

    return None

# =====
# ACCURACY FUNCTION
# =====

def accuracy(tree, test_data):

    correct = 0

    for row in test_data:

        prediction = predict(tree, row)

```

```

        if prediction == row[-1]:
            correct += 1

    return (correct / len(test_data)) * 100

# =====
# TREE DEPTH FUNCTION
# =====

def tree_depth(tree):

    if not isinstance(tree, dict):
        return 0

    feature = list(tree.keys())[0]

    depths = []

    for subtree in tree[feature].values():
        depths.append(tree_depth(subtree))

    return 1 + max(depths)

# =====
# DATASET 1 : SIMPLE DATASET
# Less Complex
# =====

dataset_simple = [

    ['Sunny', 'Hot', 'No'],
    ['Sunny', 'Cool', 'No'],
    ['Rainy', 'Cool', 'Yes'],
    ['Rainy', 'Hot', 'Yes'],
    ['Cloudy', 'Hot', 'Yes'],
    ['Cloudy', 'Cool', 'Yes']

]

# =====
# DATASET 2 : COMPLEX DATASET
# More Features + Noise
# =====

```

```

dataset_complex = [

    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Strong', 'No']

]

# =====
# SPLIT DATA INTO TRAIN AND TEST
# =====

def split_dataset(data, train_ratio=0.7):

    random.shuffle(data)

    split_index = int(len(data) * train_ratio)

    train_data = data[:split_index]

    test_data = data[split_index:]

    return train_data, test_data

# =====
# EXPERIMENT FUNCTION
# =====

def run_experiment(dataset, dataset_name):

    print("\n=====")
    print("DATASET :", dataset_name)
    print("=====")

```



```

train_data, test_data = split_dataset(dataset)

feature_count = len(dataset[0]) - 1

features = list(range(feature_count))

# -----
# UNDERFITTING CASE
# Small Depth
# -----

print("\nUNDERFITTING MODEL (MAX DEPTH = 1)")

tree_underfit = id3(train_data, features, max_depth=1)

train_acc_under = accuracy(tree_underfit, train_data)

test_acc_under = accuracy(tree_underfit, test_data)

print("Training Accuracy :", round(train_acc_under, 2), "%")
print("Testing Accuracy :", round(test_acc_under, 2), "%")
print("Tree Depth      :", tree_depth(tree_underfit))

# -----
# NORMAL MODEL
# Medium Depth
# -----

print("\nBALANCED MODEL (MAX DEPTH = 3)")

tree_balanced = id3(train_data, features, max_depth=3)

train_acc_bal = accuracy(tree_balanced, train_data)

test_acc_bal = accuracy(tree_balanced, test_data)

print("Training Accuracy :", round(train_acc_bal, 2), "%")
print("Testing Accuracy :", round(test_acc_bal, 2), "%")
print("Tree Depth      :", tree_depth(tree_balanced))

# -----
# OVERFITTING CASE
# Unlimited Depth
# -----

```

```

print("\nOVERFITTING MODEL (FULL TREE)")

tree_overfit = id3(train_data, features)

train_acc_over = accuracy(tree_overfit, train_data)

test_acc_over = accuracy(tree_overfit, test_data)

print("Training Accuracy :", round(train_acc_over, 2), "%")
print("Testing Accuracy :", round(test_acc_over, 2), "%")
print("Tree Depth      :", tree_depth(tree_overfit))

# =====
# MAIN PROGRAM
# =====

print("=====")
print("ID3 ALGORITHM PERFORMANCE ANALYSIS")
print("=====")

run_experiment(dataset_simple, "Simple Dataset")

run_experiment(dataset_complex, "Complex Dataset")

print("\n=====")
print("ANALYSIS COMPLETED")
print("=====")

```

7. **To examine different types of machine learning approaches (Supervised, Unsupervised, Semi-supervised, and Reinforcement Learning) by setting up a basic classification problem and exploring how each type**

```

# -----
# SAMPLE DATA
# -----

X = [[1], [2], [3], [8], [9], [10]]
y = [0, 0, 0, 1, 1, 1]

# =====
# 1. SUPERVISED LEARNING
# =====

```

```

print("\nSUPERVISED LEARNING")

# Simple prediction using labeled data
def supervised_predict(value):

    if value < 5:
        return 0
    else:
        return 1

print("Prediction for 2 :", supervised_predict(2))
print("Prediction for 9 :", supervised_predict(9))

# =====
# 2. UNSUPERVISED LEARNING
# =====

print("\nUNSUPERVISED LEARNING")

# Simple clustering without labels
for value in X:

    if value[0] < 5:
        cluster = "Cluster A"
    else:
        cluster = "Cluster B"

    print(value[0], "->", cluster)

# =====
# 3. SEMI-SUPERVISED LEARNING
# =====

print("\nSEMI-SUPERVISED LEARNING")

# Some data labeled, some unlabeled
labeled_data = {1: 0, 9: 1}

unlabeled = [2, 3, 8, 10]

for value in unlabeled:

    if value < 5:

```

```

        predicted_label = 0
    else:
        predicted_label = 1

    print("Value :", value,
          "Predicted Label :", predicted_label)

# =====
# 4. REINFORCEMENT LEARNING
# =====

print("\nREINFORCEMENT LEARNING")

score = 0

actions = ["Correct", "Wrong", "Correct"]

for action in actions:

    if action == "Correct":
        score += 10
        print("Reward +10")
    else:
        score -= 5
        print("Penalty -5")

print("Final Score :", score)

```

- 8. To understand how Find-S and Candidate Elimination algorithms search through the hypothesis space in concept learning tasks, and to observe the role of inductive bias in shaping the learned concept.**

```

# -----
# DATASET
# Attendance, Assignment -> Pass
# -----

data = [

    ['High', 'Complete', 'Yes'],
    ['Low', 'Complete', 'No'],
    ['High', 'Incomplete', 'Yes'],
    ['High', 'Complete', 'Yes']

]

```

```

# =====
# FIND-S ALGORITHM
# =====

print("\nFIND-S ALGORITHM")

hypothesis = ['0', '0']

for row in data:

    if row[-1] == 'Yes':

        for i in range(len(hypothesis)):

            if hypothesis[i] == '0':
                hypothesis[i] = row[i]

            elif hypothesis[i] != row[i]:
                hypothesis[i] = '?'

print("Final Hypothesis :", hypothesis)

# =====
# CANDIDATE ELIMINATION
# =====

print("\nCANDIDATE ELIMINATION")

S = ['0', '0']
G = ['?', '?']

for row in data:

    if row[-1] == 'Yes':

        for i in range(len(S)):

            if S[i] == '0':
                S[i] = row[i]

            elif S[i] != row[i]:
                S[i] = '?'

    else:

```

```

    for i in range(len(G)):

        if row[i] != S[i]:
            G[i] = S[i]

print("Specific Boundary S :", S)
print("General Boundary G :", G)

# =====
# INDUCTIVE BIAS
# =====

print("\nINDUCTIVE BIAS")

print("Find-S learns the most specific concept.")
print("Candidate Elimination keeps both")
print("specific and general boundaries.")

```

9.To go through all stages of a real-life machine learning project, from data collection to model fine-tuning, using a regression dataset like the "California Housing Prices."

```
import numpy as np
```

```

# =====
# CALIFORNIA HOUSING DATASET
# =====

# Features:
# [MedInc, HouseAge, AveRooms, AveBedrms,
# Population, AveOccup, Latitude, Longitude]

X = np.array([

    [8.3252, 41, 6.9841, 1.0238, 322, 2.5556, 37.88, -122.23],
    [8.3014, 21, 6.2381, 0.9719, 2401, 2.1098, 37.86, -122.22],
    [7.2574, 52, 8.2881, 1.0734, 496, 2.8023, 37.85, -122.24],

```

```
[5.6431, 52, 5.8174, 1.0731, 558, 2.5479, 37.85, -122.25],  
[3.8462, 52, 6.2819, 1.0811, 565, 2.1815, 37.85, -122.25],  
[4.0368, 52, 4.7617, 1.1036, 413, 2.1399, 37.85, -122.25],  
[3.6591, 52, 4.9319, 0.9510, 1094, 2.1284, 37.84, -122.25],  
[3.1200, 52, 4.7975, 1.0612, 1157, 1.7883, 37.84, -122.25],  
[2.0804, 42, 4.2941, 1.1176, 1206, 2.0269, 37.84, -122.26],  
[3.6912, 52, 4.9706, 0.9902, 1551, 2.1723, 37.84, -122.26]
```

```
)
```

```
# Target variable:
```

```
# Median House Value
```

```
y = np.array([
```

```
    4.526,
```

```
    3.585,
```

```
    3.521,
```

```
    3.413,
```

```
    3.422,
```

```
    2.697,
```

```
    2.992,
```

```
    2.414,
```

```
    2.267,
```

```
    2.611
```

```
)
```

```
# =====
```

```
# DISPLAY DATASET
```

```
# =====
```

```
print("===== CALIFORNIA HOUSING DATASET =====\n")
```

```

for i in range(len(X)):
    print("House", i + 1)
    print("Features :", X[i])
    print("Price   :", y[i])
    print()

# =====

# TRAIN-TEST SPLIT

# =====

# First 8 rows for training
X_train = X[:8]
y_train = y[:8]

# Last 2 rows for testing
X_test = X[8:]
y_test = y[8:]

print("Training Samples :", len(X_train))
print("Testing Samples  :", len(X_test))

# =====

# FEATURE SCALING

# =====

mean = np.mean(X_train, axis=0)
std = np.std(X_train, axis=0)

X_train_scaled = (X_train - mean) / std
X_test_scaled = (X_test - mean) / std

```



```

# =====

# ADD BIAS COLUMN

# =====

ones_train = np.ones((X_train_scaled.shape[0], 1))
ones_test = np.ones((X_test_scaled.shape[0], 1))

X_train_final = np.hstack((ones_train, X_train_scaled))
X_test_final = np.hstack((ones_test, X_test_scaled))

# =====

# TRAIN LINEAR REGRESSION MODEL

# =====

# Normal Equation:
#  $\Theta = (X^T X)^{-1} X^T y$ 

X_transpose = X_train_final.T

theta = np.linalg.inv(
    X_transpose.dot(X_train_final)
).dot(X_transpose).dot(y_train)

print("\n===== MODEL TRAINED SUCCESSFULLY =====\n")

print("Model Parameters:\n")
print(theta)

# =====

# MAKE PREDICTIONS

```

```

# =====

predictions = X_test_final.dot(theta)

# =====
# : MODEL EVALUATION
# =====

# Mean Squared Error

mse = np.mean((y_test - predictions) ** 2)

# Root Mean Squared Error

rmse = np.sqrt(mse)

# R2 Score

ss_total = np.sum((y_test - np.mean(y_test)) ** 2)
ss_residual = np.sum((y_test - predictions) ** 2)

r2 = 1 - (ss_residual / ss_total)

# =====

# DISPLAY RESULTS
# =====

print("\n===== MODEL PERFORMANCE =====\n")

print("Mean Squared Error :", round(mse, 4))
print("Root Mean Squared Error :", round(rmse, 4))

```

```
print("R2 Score :", round(r2, 4))
```

```
# =====
```

```
# ACTUAL VS PREDICTED
```

```
# =====
```

```
print("\n===== ACTUAL VS PREDICTED =====\n")
```

```
for i in range(len(predictions)):
```

```
    print("Actual Price   :", y_test[i])
```

```
    print("Predicted Price :", round(predictions[i], 3))
```

```
    print()
```

```
# =====
```

```
# FEATURE IMPORTANCE
```

```
# =====
```

```
feature_names = [
```

```
    "Bias",
```

```
    "Median Income",
```

```
    "House Age",
```

```
    "Average Rooms",
```

```
    "Average Bedrooms",
```

```
    "Population",
```

```
    "Average Occupancy",
```

```
    "Latitude",
```

```
    "Longitude"
```

```
]
```

```
print("\n===== FEATURE WEIGHTS =====\n")
```

```
for i in range(len(theta)):
    print(feature_names[i], "=", round(theta[i], 4))
```

10. To perform binary and multiclass classification on the MNIST dataset, analyze performance metrics, and perform error analysis.

```
# =====
# CREATE SIMPLE MNIST-LIKE DATASET
# =====
```

```
import numpy as np
```

```
# Each row represents a simple image pattern
```

```
# Values:
```

```
# 0 = White pixel
```

```
# 1 = Black pixel
```

```
# 4x4 simplified digit patterns
```

```
X = np.array([
```

```
    # Digit 0
```

```
    [1,1,1,1,
```

```
    1,0,0,1,
```

```
    1,0,0,1,
```

```
    1,1,1,1],
```

Digit 1

[0,1,0,0,

1,1,0,0,

0,1,0,0,

1,1,1,0],

Digit 2

[1,1,1,0,

0,0,1,0,

1,1,1,0,

1,0,0,0],

Digit 3

[1,1,1,0,

0,0,1,0,

1,1,1,0,

0,0,1,0],

Digit 4

[1,0,1,0,

1,0,1,0,

1,1,1,0,

0,0,1,0]

)

```

# Labels for digits

y = np.array([0,1,2,3,4])


# =====

# DISPLAY DATASET

# =====


print("===== SIMPLE MNIST-LIKE DATASET =====\n")


for i in range(len(X)):


    print("Digit Label :", y[i])


    # Convert 1D vector into 4x4 image

    image = X[i].reshape(4,4)


    print(image)

    print()


# =====

# DATASET INFORMATION

# =====


print("Dataset Shape :", X.shape)

print("Number of Labels :", len(y))

```

```
# =====  
# END OF PROGRAM  
# =====
```